

Projet linfo 1104

Pedro Cafi, Pietro Gavazzi

May 2021

Table des matières

1	Introduction	3
2	Structures Employées	4
2.1	Structure en entrée	4
2.2	Arbre de décision	4
3	Construction de l'arbre des questions	5
3.1	Fonctionnement de base	5
3.2	Extensions Incertitude dans la base de données	5
4	Navigation dans l'arbre des questions	6
4.1	Navigation sans bouton oops	6
4.1.1	code	6
4.1.2	commentaires du code	6
4.2	ajout de l'extension "Oops"	7
4.2.1	code	7
4.2.2	commentaires du code	7
A	Code du projet	9
A.1	code sans extensions	9
A.2	code avec extensions	12
B	Ancien code du projet	15
B.1	ancienne fonction "game driver"	15
B.1.1	code ancienne fonction	15
B.1.2	points faibles de l'ancienne fonction	

1 Introduction

Nous avons coder un programme permettant de poser des questions a l'utilisateur pour trouver le personnage auquel il pense avec ses réponses.

Nous avons de plus rajouter deux extensions :

- la possibilité de revenir en arrière d'une question (bouton oops)
- la possibilité de créer un arbre de décisions optimal même lorsque les personnages possèdent un nombre de champs différents

2 Structures Employées

notre code utilise deux types de structures de données :

- une liste de records comme structure en entrée contenant tous les personnages ainsi que les questions qu'on peut leur poser, et les réponses à ces questions
- pour naviguer dans le jeu avec la fonctions "GameDriver" la structure en entrée est réarrangé en un arbre de décision (un record) comme conseillé dans l'énoncé par la fonction "TreeBuilder"

2.1 Structure en entrée

```
1
2 [
3 character('Harry Potter'
4 'Est-ce que c\'est une fille ?':false
5 'A-t-il des cheveux noirs ?':true
6 'Porte-t-il des lunettes ?':true
7 'A-t-il des cheveux roux ?':false
8 )
9 character('Ron Weasley'
10 'Est-ce que c\'est une fille ?':false
11 'A-t-il des cheveux noirs ?':false
12 'Porte-t-il des lunettes ?':false
13 'A-t-il des cheveux roux ?':true
14 )
15 character('Hermione Granger'
16 'Est-ce que c\'est une fille ?':true
17 'A-t-il des cheveux noirs ?':false
18 'Porte-t-il des lunettes ?':false
19 'A-t-il des cheveux roux ?':false
20 )
21 character('Ginny Weasley'
22 'Est-ce que c\'est une fille ?':true
23 'A-t-il des cheveux noirs ?':false
24 'Porte-t-il des lunettes ?':false
25 'A-t-il des cheveux roux ?':true
26 )
27 character('Minerva McGonagall'
28 'Est-ce que c\'est une fille ?':true
29 'A-t-il des cheveux noirs ?':false
30 'Porte-t-il des lunettes ?':true
31 'A-t-il des cheveux roux ?':false
32 )
33 character('Severus Rogue'
34 'Est-ce que c\'est une fille ?':false
35 'A-t-il des cheveux noirs ?':true
36 'Porte-t-il des lunettes ?':false
37 'A-t-il des cheveux roux ?':false
38 )
39 ]
```

2.2 Arbre de décision

```
1
2 question (1:'Est-ce que c\'est une fille ?'
3   false:question(1:'A-t-il des cheveux noirs ?'
4     false:leaf(['Ron Weasley'])
5     true:question(1:'Porte-t-il des lunettes ?'
6       false:leaf(['Severus Rogue'])
7       true:leaf(['Harry Potter'])))
8   true:question(1:'A-t-il des cheveux roux ?'
9     false:question(1:'Porte-t-il des lunettes ?'
10       false:leaf(['Hermione Granger'])
11       true:leaf(['Minerva McGonagall'])))
12     true:leaf(['Ginny Weasley'])
```

chaque élément *< tree >* peut avoir une de ces deux formes particulières
soit c'est un question(1:"la question" true :< tree > false :< tree >),
soit c'est un leaf(1:["personnage"])

3 Construction de l'arbre des questions

3.1 Fonctionnement de base

Afin de construire l'arbre de décisions, nous avons utilisé l'algorithme "arbre presque optimal" décrit dans l'énoncé du projet. Ainsi nous avons donc divisé la construction de l'arbre en un ensemble de sous fonctions reprenant chacune des étapes de l'algorithme.

Nous avons tout d'abord créé une méthode **NumberOfTrueFalseOfQuestions** qui compte le nombre de réponse vrai ou fausse à chaque question et qui par la suite va retourner une liste record (Car il est plus simple d'accéder aux informations lorsqu'on connaît les champs) de ce type `valueQuestion(q : Q v : CountTrue f : CountFalse)`. `CountFalse` et `CountTrue` sont 2 mini fonction interne qui compte le nombre de vrai/faux.

La seconde étape fut de sélectionner à l'aide de **SelectQuestion** la question avec la plus faible différence de réponse vrai et fausse parmi la liste de record créée à l'étape précédente. Après avoir sélectionné la question, il a suffi d'établir une liste de personnes répondant Vrai **AnswerYesToQ** de même qu'une liste de personne répondant faux à la question **AnswerFalseToQ**.

La dernière étape fut de construire l'arbre à l'aide des sous méthodes mentionnées aux étapes précédente. Pour ce faire nous avons d'abord pris la première question sélectionnée comme racine du sous arbre ensuite supprimé cette question de la liste des questions avec **DeleteQuestion**. Ensuite il a fallu vérifier que la liste des réponses vrai/fausse était toujours séparable (càd qu'il reste encore des questions ou les personnages restant répondent différemment). Dans le cas où une(toutes) liste(s) était(étaient) toujours séparable, il ne restait qu'à appliquer le même procédé de manière récursive aux listes restantes pour chacun des sous arbres. Dans le cas où une liste n'est plus séparable, nous avons uniquement retourné `leaf([ListOfNames])` où `ListOfNames` est une fonction qui retourne uniquement une liste contenant les noms des personnages restant sans les autres informations.

3.2 Extensions Incertitude dans la base de données

Pour cette extension, il était uniquement nécessaire de modifier quelque peu le code de la création de l'arbre. Le problème étant que désormais il n'était plus assuré que les personnages possèdent le même nombre de champs. Une erreur survenait donc lorsqu'on essayait d'accéder à une question que le personnage ne possédait pas dans sa liste de question. Il suffisait donc d'attraper ces erreurs à l'aide d'un try catch et d'attribuer aucun vrai ou faux lorsque l'on compte le nombre de vrai ou faux et garder le nom du personnage dans la liste des personnes répondant vrai ainsi que dans la liste des personnes répondant faux. De plus comme il était possible que les personnages possèdent des champs différents il fallait également fournir à la méthode `NumberOfTrueOfQuestions` la liste contenant le nombre de questions maximum pour séparer le plus possible les personnages. Ce nombre de question maximum est calculé avec la méthode supplémentaire **BiggestListOfQ**.

4 Navigation dans l'arbre des questions

4.1 Navigation sans bouton oops

4.1.1 code

```
1
2 fun {GameDriver Tree}
3     Result
4
5     in
6         local
7             fun {Navigate Tree}
8                 case Tree
9                 of leaf(1:A) then {ProjectLib.found A}
10                [] question(1:A true:B false:C) then
11                    {Navigate Tree.{ProjectLib.askQuestion A}}
12                end
13            end
14            in
15                Result = {Navigate Tree}
16            end
17            if Result == false then
18                % Arf ! L'algorithme s'est trompé !
19                {Print 'Je me suis trompé \n'}
20                {Print {ProjectLib.surrender}}
21
22                % warning, Browse do not work in noGUI mode
23                {Print {ProjectLib.askQuestion 'A-t-il des cheveux roux ?'}}
24            else
25                {Print Result}
26            end
27
28            % Toujours renvoyer unit
29            unit
30        end
```

4.1.2 commentaires du code

le code est composé de d'une fonction principale, `GameDriver`, qui initialise la variable `Result` avec la fonction `Navigate`. Si `result = false`, alors il affiche la fenêtre "surrender", sinon il revient au menu de départ.

La fonction `Navigate` elle sert à naviguer dans l'arbre avec les réponses de l'utilisateur et est une fonction récursive :

Le but de la navigation est de descendre un arbre de décision (structure présentée dans la section structure). Chaque tours nous posons la question à la clé 1 du record, ensuite selon la réponse de l'utilisateur on descend encore plus dans l'arbre en prenant l'arbre à la clé `true` ou celui à la clé `false` jusqu'à ce que nous atteignons une structure `leaf([personnage])`, là nous renvoyons le personnage trouvé.

Navigation est un algorithme récursif qui à chaque fois visite l'arbre qu'on lui donne en entrée, à chaque récursion nous regardons si `Tree` est de la forme "question" ou "leaf", s'il est de la forme "leaf" nous renvoyons la réponse, s'il est de la forme "question" nous visitons l'arbre à la clé `true` ou `false`, selon la réponse du joueur.

4.2 ajout de l'extension "Oops"

4.2.1 code

```
1 fun {GameDriver Tree}
2   Result
3   in
4     local
5       fun {Navigate Tree FirstQuestion}
6         local
7           AnswerThis
8           AnswerNext
9         in
10          case Tree
11            of leaf(1:A) then {ProjectLib.found A}
12            [] question(1:A true:B false:C) then
13              AnswerThis = {ProjectLib.askQuestion Tree.1}
14              if AnswerThis == 'oops' then
15                if FirstQuestion then {Navigate Tree true}
16                else 'oops' end
17              else
18                AnswerNext = {Navigate Tree.AnswerThis false}
19                if AnswerNext == 'oops' then {Navigate Tree FirstQuestion}
20                else AnswerNext end
21              end
22            end
23          end
24        end
25      in
26        Result = {Navigate Tree true}
27      end
28      if Result == false then
29        % Arf ! L'algorithme s'est trompé !
30        {Print 'Je me suis trompé \n'}
31        {Print {ProjectLib.surrender}}
32
33        % warning, Browse do not work in noGUI mode
34        {Print {ProjectLib.askQuestion 'A-t-il des cheveux roux ?'}}
35
36      else
37        {Print Result}
38      end
39
40      % Toujours renvoyer unit
41      unit
42    end
```

4.2.2 commentaires du code

Défis en plus par apport au code sans extensions

L'extension "oops" apporte un nouveaux défi par apport à la version de base du jeu. En effet il ne faut dorénavant plus seulement parcourir l'arbre en descendant mais aussi avoir la possibilité de remonter le code si l'utilisateur pousse sur le bouton "oops".

Fonctionnement du code

Par rapport au code sans extension seule la fonction Navigate de GameDriver change. La fonction reste récursive, mais la signature de la fonction change : nous rajoutons un argument booléen FirstQuestion qui indiquera qu'il s'agit de la première fois que la fonction est appelée.

Nous avons toujours notre "case of Tree", seulement que maintenant, pour le deuxième pattern, nous stockons la réponse à la question dans la variable AnswerThis.

Si la valeur d'AnswerThis vaut 'oops' cela veut dire que l'utilisateur souhaite revenir à la question précédente, nous regardons alors si l'utilisateur répond 'oops' à la première question, si c'est le cas nous lui reposons la première question, sinon nous renvoyons 'oops' pour que ce cas soit géré par l'itération précédente de Navigate.

Si la valeur d'AnswerThis ne vaut pas 'oops', et donc vaut soit true soit false, nous poursuivons avec l'itération de la fonction sur le sous-arbre à la clé AnswerThis, mais cette fois ci en stockant la valeur retournée par

Navigate dans la variable AnswerNext. De plus nous mettons lui donnons comme argument pour FirstQuestion false.

Nous regardons si AnswerNext vaut oops, si c'est le cas c'est que l'utilisateur veut qu'on lui repose la question, nous ré appelons donc la fonction Navigate sur le Tree.
Sinon c'est que l'on a trouvé le personnage, que nous retournons.

Points faibles de la fonction

Le fait que nous terminons une récursion quand l'utilisateur utilise l'option 'oops' fait que l'on a limité les risques de stack-overflow, malgré tout le problème subsiste si l'utilisateur utilise trop souvent l'option pour la première question.

A Code du projet

A.1 code sans extensions

```
1
2 functor
3 import
4   ProjectLib
5   Browser
6   OS
7   System
8   Application
9 define
10  CWD = {Atom.toString {OS.getCWD}}#"/"
11  Browse = proc {$ Buf} {Browser.browse Buf} end
12  Print = proc{$ S} {System.print S} end
13  Args = {Application.getArgs record('nogui'(single type:bool default:false optional:true)
14        'db'(single type:string default:CWD#"database.txt"))}
15
16 in
17  local
18    NoGUI = Args.'nogui'
19    DB = Args.'db'
20    ListOfCharacters = {ProjectLib.loadDatabase file Args.'db'}
21    NewCharacter = {ProjectLib.loadCharacter file CWD#"new_character.txt"}
22    % Vous devez modifier le code pour que cette variable soit
23    % assign un argument
24    ListOfAnswersFile = CWD#"test_answers.txt"
25    ListOfAnswers = {ProjectLib.loadCharacter file CWD#"test_answers.txt"}
26    %-----
27    fun {TreeBuilder Database}
28      local
29        fun{BuildTree L Lq}
30          R1 R2 Ltrue Lfalse Nlq SpV SpF
31          fun{NumberOfTrueFalseOfQuestions L N}           %Methode qui permet de compter le
nombre de reponse vrai ou fausse      chaque questions
32          fun{CountFalse L Q F}
33            case L of H|T then try if H.Q == false then {CountFalse T Q F+1}
34              else {CountFalse T Q F} end catch error(...) then {CountFalse T Q F} end
35            [] nil then F
36          end
37          end
38          fun{CountTrue L Q V}
39            case L of H|T then try if H.Q == true then {CountTrue T Q V+1}
40              else {CountTrue T Q V} end catch error(...) then {CountTrue T Q V} end
41            [] nil then V
42          end
43          end
44
45      in
46
47          case N of H|T then valueQuestion(q:H v:{CountTrue L H 0} f:{CountFalse L H 0})|{
48            NumberOfTrueFalseOfQuestions L T}
49          [] nil then nil end
50          end
51          fun{SelectQuestion L Q A} %Methode qui permet de choisir la question avec la
52          difference la plus petite de vrai ou faux comme encourag dans l'algo
53          case L of H|T then if H.v > H.f then if H.v-H.f < A then {SelectQuestion T H.q H.v-
54            H.f}
55              else {SelectQuestion T Q A} end
56            else if H.f-H.v < A then {SelectQuestion T H.q H.f-H.v}
57            else {SelectQuestion T Q A } end
58            end
59            []nil then Q
60          end
61          end
62          fun{AnswerYesToQ L Q} %Methode qui Liste les personnes qui ont repondu vrai la
63          question
64          case L of H|T then try if H.Q==true then H|{AnswerYesToQ T Q}
65            else {AnswerYesToQ T Q}end catch error(...) then H|{AnswerYesToQ T Q}end
66          []nil then nil
67          end
68          end
69        end
70      end
71    end
72  end
73 end
```

```

65     fun{AnswerFalseToQ L Q} %Methode qui Liste les personnes qui ont repondu faux la
question
66         case L of H|T then try if H.Q==false then H|{AnswerFalseToQ T Q}
67             else {AnswerFalseToQ T Q}end catch error(...) then H|{AnswerFalseToQ T Q} end
68         []nil then nil
69         end
70     end
71     fun{DeleteQuestion L Q} %Methode qui supprime la question utilis dans la racine dans
l'ensemble des questions
72         case L of H|T then if H \= Q then H|{DeleteQuestion T Q}
73             else {DeleteQuestion T Q} end
74         []nil then nil end
75     end
76     fun{ListOfNames L} %Methode qui etabli un liste avec uniquement les noms des
personnes
77         case L of H|T then H.1|{ListOfNames T}
78         []nil then nil
79         end
80     end
81     fun{StilSeparable L} %Methode qui verifie qu'il est toujours possible de separer l'
ensemble en un ensemble plus petits
82         case L of H|T then if (H.v>0 andthen H.f>0) then true
83             else {StilSeparable T} end
84         []nil then false end
85     end
86     in
87
88     R1={NumberOfTrueFalseOfQuestions L Lq}
89     R2={SelectQuestion R1 R1.1.q 100}
90     Ltrue={AnswerYesToQ L R2}
91     Lfalse={AnswerFalseToQ L R2}
92     Nlq={DeleteQuestion Lq R2}
93     SpV={StilSeparable{NumberOfTrueFalseOfQuestions Ltrue Nlq}}
94     SpF={StilSeparable{NumberOfTrueFalseOfQuestions Lfalse Nlq}}
95
96     if (Nlq \=nil andthen SpV==true andthen SpF==true) then question(R2 true:{BuildTree
Ltrue Nlq} false:{BuildTree Lfalse Nlq})
97     else if (Nlq \=nil andthen SpV==true andthen SpF==false) then question(R2 true:{
BuildTree Ltrue Nlq} false:leaf({ListOfNames Lfalse}))
98     else if (Nlq \=nil andthen SpV==false andthen SpF==true) then question(R2 true:leaf({
ListOfNames Ltrue}) false:{BuildTree Lfalse Nlq})
99     else question(R2 true:leaf({ListOfNames Ltrue}) false:leaf({ListOfNames Lfalse
})) end
100     end
101     end
102     end
103     end
104     fun{BiggestListOfQ L A}
105         case L of H|T then if {Length{Arity H}.2}>{Length A} then {BiggestListOfQ T {Arity H
}.2}
106             else{BiggestListOfQ T A} end
107         []nil then A end
108     end
109     in
110     {BuildTree Database {BiggestListOfQ Database nil}}
111     end
112     end
113
114 %-----
115
116     fun {GameDriver Tree}
117         Result
118     in
119         local
120         fun {Navigate Tree}
121             case Tree
122             of leaf(1:A) then {ProjectLib.found A}
123             [] question(1:A true:B false:C) then
124                 {Navigate Tree.{ProjectLib.askQuestion A}}
125             end
126         end
127     in
128         Result = {Navigate Tree}
129     end

```

```

130         if Result == false then
131             % Arf ! L'algorithme s'est trompé !
132             {Print 'Je me suis trompé \n'}
133             {Print {ProjectLib.surrender}}
134
135             % warning, Browse do not work in noGUI mode
136             {Print {ProjectLib.askQuestion 'A-t-il des cheveux roux ?'}}
137
138         else
139             {Print Result}
140         end
141
142         % Toujours renvoyer unit
143         unit
144     end
145 in
146     {ProjectLib.play opts(characters:ListOfCharacters driver:GameDriver
147                           noGUI:NoGUI builder:TreeBuilder
148                           autoPlay:ListOfAnswers newCharacter:NewCharacter
149                           oopsButton:false )}
150     {Application.exit 0}
151 end
152 end

```

A.2 code avec extensions

```

1 functor
2 import
3   ProjectLib
4   Browser
5   OS
6   System
7   Application
8 define
9   CWD = {Atom.toString {OS.getCWD}}#"/"
10  Browse = proc {$ Buf} {Browser.browse Buf} end
11  Print = proc {$ S} {System.print S} end
12  Args = {Application.getArgs
13  record(
14    'nogui'(single type:bool default:false optional:true)
15    'db'(single type:string default:CWD#"database.txt")
16  )}
17
18 in
19   local
20     NoGUI = Args.'nogui'
21     DB = Args.'db'
22     ListOfCharacters = {ProjectLib.loadDatabase file Args.'db'}
23     NewCharacter = {ProjectLib.loadCharacter file CWD#"new_character.txt"}
24     % Vous devez modifier le code pour que cette variable soit assign un argument
25     ListOfAnswersFile = CWD#"test_answers.txt"
26     ListOfAnswers = {ProjectLib.loadCharacter file CWD#"test_answers.txt"}
27 % -----
28   fun {TreeBuilder Database}
29     local
30       fun{BuildTree L Lq}
31         R1 R2 Ltrue Lfalse Nlq SpV SpF
32         fun{NumberOfTrueFalseOfQuestions L N} %Methode qui permet de compter le
33         nombre de reponse vrai ou fausse   chaque questions
34         fun{CountFalse L Q F}
35           case L of H|T then try if H.Q == false then {CountFalse T Q F+1}
36             else {CountFalse T Q F} end catch error(...) then {CountFalse T Q F} end
37           [] nil then F
38         end
39         fun{CountTrue L Q V}
40           case L of H|T then try if H.Q == true then {CountTrue T Q V+1}
41             else {CountTrue T Q V} end catch error(...) then {CountTrue T Q V} end
42           [] nil then V
43         end
44       end
45     in
46       case N of H|T then valueQuestion(q:H v:{CountTrue L H 0} f:{CountFalse L H 0})|{
47       NumberOfTrueFalseOfQuestions L T}
48       [] nil then nil end
49     end
50     fun{SelectQuestion L Q A} %Methode qui permet de choisir la question avec la
51     difference la plus petite de vrai ou faux comme encourag dans l'algo
52     case L of H|T then if H.v > H.f then if H.v-H.f < A then {SelectQuestion T H.q H.v-
53     H.f}
54       else {SelectQuestion T Q A} end
55       else if H.f-H.v < A then {SelectQuestion T H.q H.f-H.v}
56       else {SelectQuestion T Q A} end
57     end
58     [] nil then Q
59   end
60   fun{AnswerYesToQ L Q} %Methode qui Liste les personnes qui ont repondu vrai la
61   question
62   case L of H|T then try if H.Q==true then H|{AnswerYesToQ T Q}
63     else {AnswerYesToQ T Q}end catch error(...) then H|{AnswerYesToQ T Q}end
64   [] nil then nil
65   end
66   fun{AnswerFalseToQ L Q} %Methode qui Liste les personnes qui ont repondu faux la
67   question

```

```

67         case L of H|T then try if H.Q==false then H|{AnswerFalseToQ T Q}
68             else {AnswerFalseToQ T Q}end catch error(...) then H|{AnswerFalseToQ T Q} end
69         []nil then nil
70     end
71 end
72 fun{DeleteQuestion L Q} %Methode qui supprime la question utilis dans la racine dans
l'ensemble des questions
73     case L of H|T then if H \= Q then H|{DeleteQuestion T Q}
74         else {DeleteQuestion T Q} end
75     []nil then nil end
76 end
77 fun{ListOfNames L} %Methode qui etabli un liste avec uniquement les noms des
personnes
78     case L of H|T then H.1|{ListOfNames T}
79     []nil then nil
80     end
81 end
82 fun{StilSeparable L} %Methode qui verifie qu'il est toujours possible de separer l'
ensemble en un ensemble plus petits
83     case L of H|T then if(H.v>0 andthen H.f>0) then true
84         else {StilSeparable T} end
85     []nil then false end
86 end
87
88
89 in
90     R1={NumberOfTrueFalseOfQuestions L Lq}
91     R2={SelectQuestion R1 R1.1.q 100}
92     Ltrue={AnswerYesToQ L R2}
93     Lfalse={AnswerFalseToQ L R2}
94     Nlq={DeleteQuestion Lq R2}
95     SpV={StilSeparable{NumberOfTrueFalseOfQuestions Ltrue Nlq}}
96     SpF={StilSeparable{NumberOfTrueFalseOfQuestions Lfalse Nlq}}
97
98     if (Nlq \=nil andthen SpV==true andthen SpF==true) then question(R2 true:{BuildTree
Ltrue Nlq} false:{BuildTree Lfalse Nlq})
99     else if (Nlq \=nil andthen SpV==true andthen SpF==false) then question(R2 true:{
BuildTree Ltrue Nlq} false:leaf({ListOfNames Lfalse}))
100     else if (Nlq \=nil andthen SpV==false andthen SpF==true) then question(R2 true:leaf({
ListOfNames Ltrue}) false:{BuildTree Lfalse Nlq})
101     else question(R2 true:leaf({ListOfNames Ltrue}) false:leaf({ListOfNames Lfalse
})) end
102 end
103 end
104 end
105 fun{BiggestListOfQ L A}
106     case L of H|T then if{Length{Arity H}.2}>{Length A} then {BiggestListOfQ T {Arity H
}.2}
107         else{BiggestListOfQ T A} end
108     []nil then A end
109 end
110 in
111     {BuildTree Database {BiggestListOfQ Database nil}}
112 end
113 end
114
115 % -----
116
117 fun {GameDriver Tree}
118     Result
119 in
120     local
121     fun {Navigate Tree FirstQuestion}
122         local
123             AnswerThis
124             AnswerNext
125         in
126             case Tree
127             of leaf(1:A) then {ProjectLib.found A}
128             [] question(1:A true:B false:C) then
129                 AnswerThis = {ProjectLib.askQuestion Tree.1}
130                 if AnswerThis == 'oops' then
131                     if FirstQuestion then {Navigate Tree true}
132                     else 'oops' end

```

```

133         else
134             AnswerNext = {Navigate Tree.AnswerThis false}
135             if AnswerNext == 'oops' then {Navigate Tree.FirstQuestion}
136             else AnswerNext end
137         end
138     end
139 end
140 end
141 in
142     Result = {Navigate Tree true}
143 end
144 if Result == false then
145     % Arf ! L'algorithme s'est trompé !
146     {Print 'Je me suis trompé \n'}
147     {Print {ProjectLib.surrender}}
148
149     % warning, Browse do not work in noGUI mode
150     {Print {ProjectLib.askQuestion 'A-t-il des cheveux roux ?'}}
151
152 else
153     {Print Result}
154 end
155
156 % Toujours renvoyer unit
157 unit
158 end
159
160 % -----
161 in
162     {ProjectLib.play opts(characters:ListOfCharacters driver:GameDriver
163                          noGUI:NoGUI builder:TreeBuilder
164                          autoPlay:ListOfAnswers newCharacter:NewCharacter
165                          oopsButton:true )}
166     {Application.exit 0}
167 end
168 end

```

B Ancien code du projet

B.1 ancienne fonction "game driver"

B.1.1 code ancienne fonction

```
1
2 fun {GameDriver Tree}
3   Result
4 in
5   local
6
7     fun {Navigate Tree List}
8       local
9         NewList
10        fun {Accumulator Tree List}
11          case List
12          of H | T then {Accumulator Tree.H T}
13          [] nil then Tree
14          end
15        end
16        fun {LastElement List}
17          case List
18          of H|nil then H
19          [] H|T then {LastElement T}
20          end
21        end
22        fun {PopTwoElements List}
23          case List
24          of nil then nil
25          [] A|nil then nil
26          [] A|B|nil then nil
27          [] A|B|C|nil then A|nil
28          [] A|B then {Append A|nil {PopTwoElements B}}
29          end
30        end
31      in
32        case {Accumulator Tree List}
33        of leaf(1:A) then {ProjectLib.found A}
34        [] question(1:A true:B false:C) then
35          NewList = {Append List {ProjectLib.askQuestion A}|nil}
36          if {LastElement NewList} == 'oops' then
37            {Navigate Tree {PopTwoElements NewList}}
38          else
39            {Navigate Tree NewList}
40          end
41        end
42      end
43    end
44  in
45    Result = {Navigate Tree nil}
46  end
47  if Result == false then
48    % Arf ! L'algorithme s'est trompé !
49    {Print 'Je me suis trompé \n'}
50    {Print {ProjectLib.surrender}}
51
52    % warning, Browse do not work in noGUI mode
53    {Print {ProjectLib.askQuestion 'A-t-il des cheveux roux ?'}}
54
55  else
56    {Print Result}
57  end
58
59  % Toujours renvoyer unit
60  unit
61 end
```

B.1.2 points faibles de l'ancienne fonction

A chaque itération l'arbre de décision doit être re-parcouru avec la fonction `Accumulator`, mais cela ne portera pas préjudice, selon nous, à l'expérience de jeu à moins d'un arbre titanesque.

Le programme aurait par contre éventuellement pu planter à cause d'un stack-overflow de la mémoire : en effet à chaque itération certaines variable (`list`, `newlist`,...) restent sauvés dans la mémoire comme résidus, et le jeu peut durer potentiellement éternellement.